
Behavioral cloning model for Pokémon VGC meta game

July 9, 2026

Calderari Eleonora, Coppola Armando, Iezzi Sofia

Abstract

The aim of this work is to develop and compare different models capable of playing a match of Pokémon Video Game Championship (VGC). The task is to predict the best pair of moves to execute during a specific turn of the match in order to secure a victory.

In particular, we present an architecture in which we implement a Decision Transformer for off-line Reinforcement Learning (RL), inspired by the work of Chen *et al.* (2021) (3).

To understand the model's adaptability across different competitive environments, we investigate various training strategies using matches from Regulation M-A and Regulation M-B. We compare baseline models trained exclusively on individual regulations against more complex approaches, including transfer learning (fine-tuning a pre-trained Regulation M-A model on Regulation M-B data) and training on a combined dataset. Finally, we compare the overall performance and predictive accuracy of the models.

1. Introduction

Pokémon VGC is the official competitive tournament organized by The Pokémon Company. Although the rules remain the same, there are different levels in which the game can be played. The most common is the online ladder, however many competitive players compete at the top of global rankings in live events.

We present below the main rules of the competition.

The match. Every battle consists of a match, divided into a different numbers of turns, depending on the match du-

Email: Calderari Eleonora, Coppola Armando, Iezzi Sofia <calderari.2024102@studenti.uniroma1.it, coppola.2003964@studenti.uniroma1.it, iezzi.2004627@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

ration. There are two players, each with their own team, consisting of six different Pokémon. Each player must select exactly four out of their six Pokémon to bring into the battle.

In this competitive format, matches are played as Double Battles. This means that each player has only two Pokémon on the field simultaneously, while the other two of the team remain on the bench.

The turns. Before a turn is started, the player has to choose the moves the Pokémon have to perform. Moves are chosen at the same time and blindly by both players, so the system does not operate sequentially, in order to preclude players from reacting to the outcome of a preceding action.

Once both players make their choices, the turn begins. Under normal field conditions, the fastest Pokémon performs its action first.

The moves. The number of moves available for a single Pokémon can be very large. They generally allow the Pokémon to attack the opponents, to protect itself or to change the features of the field. Before every match, the player has to select four moves for each of their Pokémon, creating his winning strategy. In Double Battle format, selecting a move also requires choosing a target among the Pokémon on the field. Moreover, instead of using a move, it is also possible to switch a Pokémon currently on the field with one alive on the bench. Therefore at each turn the player can choose among five different options for each active Pokémon.

The Pokémon. Pokémon are highly different and their features allow players to build up different strategies. When a Pokémon is hit by an attacking move its HP (Hit Points) decreases and it remains on the field as long as it remains conscious. When a Pokémon is fainted, the player is forced to replace it with one alive from the bench.

When a player has no more alive Pokémon remaining on the field or on the bench, the match is ended and the opponent is the winner.

2. Dataset

The dataset comprises recent matches extracted from the public replays on Pokémon Showdown, the most popular online battle simulator. Specifically, we considered matches from the current 2026 competitive season and filtered for games played by users with an Elo rating greater than 1800. While a larger dataset could have been constructed by including lower-Elo players, we noticed that such an approach would introduced numerous sub-optimal moves. This would have produced a highly noisy dataset.

Instead of using an older format with more battles, we also decided to refer only to the most recent format due to the restricted number of Pokémon and items.

We partitioned the dataset into two subsets corresponding to distinct official regulations: Regulation M-A (reg_m-A) and Regulation M-B (reg_m-B). The former was the initial ruleset active until June 17, 2026, whereas the latter is the currently active format. The transition to Regulation M-B introduced a significant mega-shift, expanding the legal pool with 22 new Pokémon and 16 new Mega Evolutions.

To enlarge the dataset and maximize information extraction from the logs, we duplicated the logs by reversing the perspectives of Player 0 and Player 1.

We tokenized each battle log by turn. In each turn, we considered all the 12 Pokémon, battlefield conditions (weather, speed boost) and the two selected actions. Each Pokémon is represented by its unique identifier and its statistics. Moreover, we track status conditions, statistics level changes and moves revealed during the battle. Since the battle is represented from the point of view of Player 0 (p1 in the log), we know the items, abilities and moves of all his Pokémon from the first token. Conversely, moves, items and abilities of the opponent’s Pokémon are revealed only after the turn in which they are first utilized.

3. Related works

AlphaGo. The application of Reinforcement Learning to complex strategic games achieved one of its greatest milestones with AlphaGo. We were inspired by the initial training phase of the model, which used human expert gameplay. However, unlike AlphaGo, we do not train the model against other version of itself, using self-play training mechanisms. Other differences between our approach and AlphaGo lie in the sequential nature of the game Go. Consequently, we adopted a different evaluation strategy: rather than testing our models against human opponents in live matches, we employed an Offline RL framework, comparing the predicted actions with moves actually selected by human world champions.

4. Architecture

We implemented the architecture of a Decision Transformer following the work of Chen *et al.* (2021) (3).

All data must be represented as tokens. We consider three types of input tokens — state, action and reward — and then merge them forming a single tensor token, used as the input of the embedding layer. We also embed a tensor for the turn that is later added to the previous tokens before going through the transformer blocks.

The state contains all the information about the 12 Pokémon, the moves that each of them can perform and the features of the battlefield. The action consists of the two moves that Player 0 chooses to execute, the user, the target and their positions on the field. While the main information about the moves are contained in the state token, the action contains just an index related to one move of the user. Finally, the reward is a single number: zero if Player 0 is the winner and one if he is the loser.

The output of the model is the predicted action. Thus the model determines the user, the targets and the two moves to perform in the current turn. We also implemented a legal action mask in order to ensure that the predicted moves would be acceptable.

To evaluate the model’s performance, we employed the Negative Log-Likelihood Loss Function (NLLLoss), which measures the discrepancy between the ground-truth actions and the model’s predicted probability distributions at each turn.

However, unlike the return-to-go mechanism in offline RL, we observed that the reward token exerts minimal influence on the model’s predictions, contrary to our initial hypothesis. The model implicitly infers the winning probability from the sequence of actions and states and propagates this estimation, even though we are conditioning on a losing trajectory (see more details in Section (6)).

Consequently, we obtained a model that more closely resembles behavioral cloning; in other words, it learns to mimic the players from the observed matches.

During the initial phases of training, we also observed significant overfitting within the first few epochs. To address this issue, we implemented a data augmentation function that permutes the order of the moves for each Pokémon. This permutation remains constant throughout all turns of a given match. Additionally, we applied a random permutation to the sequence of the 12 Pokémon within the state token. By shuffling the data in this manner, we prevented the model from memorizing arbitrary positional patterns, compelling it instead to develop a robust understanding of the intrinsic properties of the moves and the Pokémon themselves.

5. Models

We trained a total of four models, all sharing a common architecture, comprising 8 transformer blocks, 12 attention heads and an embedding dimension of 384, resulting in 17.46 million trainable parameters. This specific configuration was determined after empirical hyperparameter tuning, evaluating different combinations of attention heads, transformer blocks and embedding dimensions. Alternative configurations yielded higher loss values and inferior accuracy. Furthermore, we tried to increase the embedding dimension of the single feature of each token, from `feat_dim=16` to `feat_dim=32`. However, experimental results indicated no significant improvements in model performance, thus we decided to choose `feat_dim=16`.

Across all configurations, the dataset was partitioned using an 80% split for the training set, with the remaining 20% allocated to validation and testing.

The first model was trained exclusively on the Regulation M-A (reg m-A) matches, whereas the second model was trained only on the Regulation M-B (reg m-B) matches. To achieve higher accuracy, a third model was developed using a transfer learning approach: we initialized the network with the weights pre-trained on the reg m-A dataset and fine-tuned it on the reg m-B matches. To preserve these pre-trained weights, the model’s backbone was frozen for the first three epochs—updating only the final transformer block and the prediction head—before unfreezing the entire network for full fine-tuning. This process utilized the AdamW optimizer with a peak learning rate of 10^{-5} , incorporating a 2-epoch linear warmup to prevent catastrophic forgetting, followed by a cosine decay schedule.

Finally, the fourth model was trained on the entirety of the dataset, combining both the reg m-A and reg m-B matches.

6. Experimental results

The four trained models were compared through a series of tests, with the resulting accuracy and loss metrics reported in the tables below (1, 2, 3). It can be observed that the fine-tuned model exhibits an accuracy comparable to that of the model trained exclusively on reg m-A, albeit with a lower loss. This demonstrates that the model trained on reg m-A had already maximized its learning of player behaviors and underlying game dynamics from the provided dataset. Furthermore, it demonstrates strong generalization capabilities, achieving an accuracy of 34% when evaluated on all reg m-B format matches. Consequently, the fine-tuning process does not significantly expand the model’s knowledge base but rather stabilizes its weights, as evidenced by the reduction in loss.

The optimal results are achieved by training the model directly on the combined dataset containing matches from both reg m-A and reg m-B. This approach yields a higher accuracy of 38.55% and a lower overall loss. The larger volume of data, coupled with the increased heterogeneity introduced by combining both formats, enables the Transformer to develop a more robust understanding of the underlying game mechanics and the specific variables encoded within the tokens.

A final experiment was conducted to evaluate the impact of the reward token. During the testing phase, we altered the matches fed to the model by inverting the reward conditioning. Specifically, we flipped the reward token—conditioning the model with a winning reward ($R = 0$) when predicting the actions of the player who ultimately lost the match, and vice versa. The aim, in this case, was to induce the model to generate a winning move. By comparing this generated prediction against the true target action, we anticipated observing a high loss and a low accuracy, which would demonstrate that the reward token actively conditions the model’s decision-making. However, both the accuracy and the loss metrics remained unchanged. This outcome validates the hypothesis that the network is primarily performing behavioral cloning. During inference, the model relies on the sequence of turns preceding the prediction step; the tokens encoding the state of this match history exert a significantly stronger influence on the model’s decisions than the single scalar bit provided by the reward token. (4)

The legend used for the experimental results is presented below

- **Metamon_A**: the model trained on reg_m-A
- **Metamon_B**: the model trained on reg_m-B
- **Metamon_BonA**: the model trained on reg_m-A then fine tuned on reg_m-B
- **Metamon_AB**: the model trained simultaneously on reg_m-A and reg_m-B

Tested on	accuracy	loss
test set reg_m-A	35.39%	2.1226
all reg_m-B DS	34.26%	2.2044

Table 1. Loss and accuracy of Metamon_A on the test set of the reg_m-A dataset compared with a zero shot test on all the reg_m-B dataset

Tested on	accuracy	loss
test set reg_m-B	36.85%	2.0870
all reg_m-A DS	35.83%	2.1129

Table 2. Loss and accuracy of Metamon_B on the test set of the reg_m-B dataset compared with a zero shot test on all the reg_m-A dataset

model	Tested on	accuracy	loss
Metamon_BonA	test reg_m-B	35.67%	2.0968
Metamon_AB	test reg_m-A+B	38.55%	1.9574

Table 3. comparison between Metamon_BonA and Metamon_AB

model	Tested on	accuracy	loss
Metamon_AB	right reward	38.55%	1.9574
Metamon_AB	flipped reward	38.56%	1.9597

Table 4. test forcing flipped reward in the _run_batch function

7. Conclusions

In conclusion, it can be stated that utilizing a larger dataset from the outset yields more satisfactory results, even outperforming the fine-tuning approach. Nevertheless, the maximum accuracy does not exceed approximately 38%, which can be attributed to the relatively limited size of even the combined dataset. To achieve superior performance, the Transformer architecture would require a substantially larger volume of data. Furthermore, in the context of behavioral cloning, where the objective is to predict the exact move selected by the player, surpassing an accuracy threshold of 40% represents a significant challenge. This limitation is largely due to the inherent element of unpredictability that characterizes human decision-making.

Future improvements and prospective research directions could involve the implementation of an incremental reward system assigned to each individual action, mirroring the original Decision Transformer architecture detailed in (3). This modification would effectively transition the project from a behavioral cloning approach to a full reinforcement learning framework. Furthermore, the development of a dedicated online simulator would be of significant interest. Such an environment would enable the trained model to engage in live matches, thereby facilitating and enhancing the reinforcement learning process through active interaction with the game state.

References

- [1] Jay Alammar. Visualizing a neural machine translation model (mechanics of seq2seq models with attention), 2018. URL: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>.
- [2] Peter Bloem. Transformers from scratch, 2019. URL: <https://peterbloem.nl/blog/transformers>.
- [3] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *arXiv preprint arXiv:2106.01345*, 2021. URL: <https://arxiv.org/abs/2106.01345>.
- [4] Jaron Collis. Glossary of deep learning: Word embedding, 2017. URL: <https://medium.com/deepeer-learning/glossary-of-deep-learning-word-embedding-f90c3cec34ca>.
- [5] Chitta Ranjan. Sequence embedding for clustering and classification, 2019. URL: <https://medium.com/data-science/sequence-embedding-for-clustering-and-classification-f816a66373fb>.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL: <https://arxiv.org/pdf/1706.03762>.

Appendix

For completeness, a reference to the online Pokémon battle simulator is provided below

<https://pokemonshowdown.com/>

More data about each Pokémon were provided from the online dataset:

<https://pokeapi.co/>

Use of AI. Artificial Intelligence tools were utilized during the initial brainstorming phase to conduct a preliminary literature review and identify relevant sources, most notably assisting in the discovery of the foundational paper on the Decision Transformer. As the project advanced, AI was employed for code refactoring and debugging. Specifically, it was leveraged to optimize the computational efficiency of the data preprocessing pipeline and the action masking

logic, as well as to obtain technical guidance on configuring the multi-GPU training environment on the Kaggle platform.

Furthermore, the application of AI was extended to the evaluation phase, where it provided valuable recommendations for some testing methodologies. Finally, AI tools were used for proofreading and linguistic refinement of this report.